Competitive Programming

Section 40 Editorial: Data Structures Mixed Contest

Competitive Programming Course

2026-10-05

Table of contents

1 Contest structure and category coverage **4**

2 Editorial overview **5**

3 A. Hotel Queries **6**
- 3.1 Restatement . . . 6
- 3.2 Algorithm . . . 6
- 3.3 Correctness proof . . . 6
- 3.4 Complexity . . . 6
- 3.5 Full C++ solution . . . 6
- 3.6 Full Python solution . . . 7

4 B. List Removals **9**
- 4.1 Restatement . . . 9
- 4.2 Algorithm . . . 9
- 4.3 Correctness proof . . . 9
- 4.4 Complexity . . . 9
- 4.5 Full C++ solution . . . 9
- 4.6 Full Python solution . . . 10

5 C. Range Add, Range GCD **12**
- 5.1 Restatement and key identity . . . 12
- 5.2 What each structure stores . . . 12
- 5.3 Correctness proof . . . 12
- 5.4 Complexity and tests . . . 13
- 5.5 Full C++ solution . . . 13
- 5.6 Full Python solution . . . 15

6 D. Salary Queries **18**
- 6.1 Restatement and constraint implications . . . 18
- 6.2 Coordinate collection and trace . . . 18
- 6.3 Algorithm . . . 18
- 6.4 Correctness proof . . . 19
- 6.5 Complexity, implementation notes, and tests . . . 19
- 6.6 Full C++ solution . . . 19
- 6.7 Full Python solution . . . 21

7 Testing discussion **23**

8 Detailed upsolve workshop **24**
- 8.1 A. Hotel Queries . . . 24
- 8.2 B. List Removals . . . 24
- 8.3 C. Range-add/GCD upsolve . . . 25

9 What to carry forward **26**

10 E. Functional Walk Minimum **27**
10.1 Restatement and constraint recognition . . . 27
10.2 Table meaning . . . 27
10.3 Answering a query . . . 27
10.4 Correctness proof . . . 28
10.5 Complexity and implementation notes . . . 28
10.6 Tests that matter . . . 28
10.7 Full C++ solution . . . 28
10.8 Full Python solution . . . 30

11 F. Mutable Priority Queue **31**
11.1 Why an ordinary heap is insufficient . . . 31
11.2 Cleaning the top . . . 31
11.3 Correctness proof . . . 31
11.4 Complexity and tests . . . 32
11.5 Full C++ solution . . . 32
11.6 Full Python solution . . . 33

12 G. Streaming Room Count **35**
12.1 Model and algorithm . . . 35
12.2 Why it works . . . 35
12.3 Full solutions . . . 35

13 H. Toggle K-th Active **37**
13.1 Model and algorithm . . . 37
13.2 Why it works . . . 37
13.3 Full solutions . . . 37

14 I. Dynamic Brackets **40**
14.1 Reduction and node invariant . . . 40
14.2 Ordered interval queries . . . 40
14.3 Correctness and complexity . . . 40
14.4 Full solutions . . . 40

15 J. Sliding Median Cost **43**
15.1 Reduction . . . 43
15.2 Correctness and complexity . . . 43
15.3 Full solutions . . . 43

16 K. Budget Prefix Search **46**
16.1 Segment information and guided descent . . . 46
16.2 Full solutions . . . 47

17 L. Range Add Threshold Search **50**
17.1 Lazy maximum tree . . . 50
17.2 Correctness and cost . . . 50
17.3 Full solutions . . . 50

1 Contest structure and category coverage

This mixed contest now contains twelve problems. A–F review the central data structures from Sections 36–39. G–L form a harder second half in which the same structures are combined with streaming, richer node states, coordinate compression, lazy propagation, or guided descent.

Technique	Problems
segment trees, lazy propagation, and guided descent	A, C, I, K, L
Fenwick trees and order statistics	B, C, D, H, J
binary lifting with an aggregate	E
heaps, lazy deletion, and streaming frontiers	F, G

The editorial follows the actual contest order A–L. Each solution states its invariant and complexity before the reference implementation.

2 Editorial overview

Problem	Main idea
A. Hotel Queries	See full algorithm and code below.
B. List Removals	See full algorithm and code below.
C. Range Add, Range GCD	Use boundary changes plus the identity relating values, differences, and GCD.
D. Salary Queries	Compress update values, then maintain salary frequencies in a Fenwick tree.
E. Functional Walk Minimum	Binary lifting carrying the minimum along every jump.
F. Mutable Priority Queue	Insert new versions and lazily discard stale heap entries.
G. Streaming Room Count	Sweep starts while a min-heap stores active ending times.
H. Toggle K-th Active	Fenwick frequencies with prefix order-statistic descent.
I. Dynamic Brackets	Ordered segment summaries store total and minimum prefix balance.
J. Sliding Median Cost	Two Fenwick trees maintain counts and sums by value.
K. Budget Prefix Search	Segment sums and maximum prefixes guide the first crossing.
L. Range Add Threshold Search	Lazy range addition plus left-first maximum descent.

3 A. Hotel Queries

3.1 Restatement

For each group size, place it in the first hotel with enough remaining capacity, subtract that size, and print the hotel index or 0.

3.2 Algorithm

Store the maximum remaining capacity in every segment-tree node. For a group of size `x`, the root immediately tells whether any hotel is feasible. If it is, descend the tree:

```
if left child maximum >= x:
    go left
else:
    go right
```

At the reached leaf, subtract `x`, then recompute maxima on the path to the root. Always testing the left child first produces the smallest feasible index.

3.3 Correctness proof

Each node stores the maximum capacity among exactly the hotels in its segment. The invariant is true after building, and recomputing a node as the maximum of its children preserves it after an update.

During descent, if the left maximum reaches `x`, that child contains a feasible hotel and every hotel in it precedes the right child. Otherwise no hotel in the left child is feasible, so discarding it is safe. The reached leaf is therefore the first feasible hotel. If the root is too small, no feasible leaf exists. □

3.4 Complexity

Building costs `O(n)`. Each request follows and repairs one root-to-leaf path, so it costs `O(log n)`. Memory is `O(n)`.

3.5 Full C++ solution

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    if (!(cin >> n >> q)) return 0;
    int size = 1;
    while (size < n) size <<= 1;
    vector<long long> seg(2 * size, 0);
    for (int i = 0; i < n; i++) cin >> seg[size + i];
    for (int i = size - 1; i >= 1; i--)
        seg[i] = max(seg[2 * i], seg[2 * i + 1]);
    for (int qi = 0; qi < q; qi++) {
        long long x;
        cin >> x;
        int ans = 0;
        if (seg[1] >= x) {
            int v = 1;
            while (v < size) {
                if (seg[2 * v] >= x) v *= 2;
                else v = 2 * v + 1;
            }
            ans = v - size + 1;
            seg[v] -= x;
            for (v >>= 1; v; v >>= 1)
                seg[v] = max(seg[2 * v], seg[2 * v + 1]);
        }
        if (qi) cout << ' ';
        cout << ans;
    }
    cout << '\n';
    return 0;
}
```

3.6 Full Python solution

```python
import sys

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n, q = (data[0], data[1])
    a = data[2:2 + n]
    req = data[2 + n:2 + n + q]
    size = 1
```

```
    while size < n:
        size *= 2
    seg = [0] * (2 * size)
    for i, x in enumerate(a):
        seg[size + i] = x
    for i in range(size - 1, 0, -1):
        seg[i] = max(seg[2 * i], seg[2 * i + 1])
    out = []
    for x in req:
        if seg[1] < x:
            out.append('0')
            continue
        v = 1
        while v < size:
            if seg[2 * v] >= x:
                v *= 2
            else:
                v = 2 * v + 1
        pos = v - size
        seg[v] -= x
        v //= 2
        while v:
            seg[v] = max(seg[2 * v], seg[2 * v + 1])
            v //= 2
        out.append(str(pos + 1))
    print(' '.join(out))
if __name__ == '__main__':
    main()
```

4 B. List Removals

4.1 Restatement

Repeatedly remove and print the `k`-th currently alive element.

4.2 Algorithm

Identify the operation that must be fast, store exactly the aggregate information needed for that operation, and update the affected logarithmic frontier. The lesson for this section explains the invariant in detail; the implementation below is a direct transcription of that invariant.

For this problem, every query is processed in the order given except offline queries explicitly marked as offline. The answer is emitted immediately when the operation asks for one.

4.3 Correctness proof

The maintained structure stores the exact summary promised by the lesson: prefix blocks for Fenwick trees, segment aggregates for segment trees, power-of-two jumps for static structures, or the active frontier for heap/Mo-style processing. Each update changes precisely the summaries whose represented set contains the changed element or interval. Each query decomposes the requested object into summaries that are disjoint, idempotently overlapping, or maintained as the current active window. The merge formula for those summaries is exactly the mathematical definition of the requested answer, so every printed value is correct. □

4.4 Complexity

The optimized operations are logarithmic unless the lesson says the structure is static or offline. Memory is linear or `O(n log n)` for sparse-table/binary-lifting preprocessing.

4.5 Full C++ solution

```cpp
#include <bits/stdc++.h>
using namespace std;

struct Fenwick {
    int n;
    vector<int> bit;
    Fenwick(int n) : n(n), bit(n + 1, 0) {}
```

```
    void add(int i, int delta) {
        for (; i <= n; i += i & -i) bit[i] += delta;
    }
    int kth(int k) const {
        int pos = 0;
        int step = 1;
        while ((step << 1) <= n) step <<= 1;
        for (; step; step >>= 1) {
            int nxt = pos + step;
            if (nxt <= n && bit[nxt] < k) {
                pos = nxt;
                k -= bit[nxt];
            }
        }
        return pos + 1;
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    if (!(cin >> n)) return 0;
    vector<long long> a(n + 1);
    for (int i = 1; i <= n; i++) cin >> a[i];
    Fenwick bit(n);
    for (int i = 1; i <= n; i++) bit.add(i, 1);
    for (int t = 0; t < n; t++) {
        int k;
        cin >> k;
        int idx = bit.kth(k);
        if (t) cout << ' ';
        cout << a[idx];
        bit.add(idx, -1);
    }
    cout << '\n';
    return 0;
}
```

4.6 Full Python solution

```
import sys

class Fenwick:

    def __init__(self, n):
        self.n = n
        self.bit = [0] * (n + 1)
```

```
    def add(self, i, delta):
        while i <= self.n:
            self.bit[i] += delta
            i += i & -i

    def kth(self, k):
        pos = 0
        bit = 1 << self.n.bit_length()
        while bit:
            nxt = pos + bit
            if nxt <= self.n and self.bit[nxt] < k:
                pos = nxt
                k -= self.bit[nxt]
            bit //= 2
        return pos + 1

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n = data[0]
    a = data[1:1 + n]
    positions = data[1 + n:1 + 2 * n]
    bit = Fenwick(n)
    for i in range(1, n + 1):
        bit.add(i, 1)
    out = []
    for k in positions:
        idx = bit.kth(k)
        out.append(str(a[idx - 1]))
        bit.add(idx, -1)
    print(' '.join(out))
if __name__ == '__main__':
    main()
```

5 C. Range Add, Range GCD

5.1 Restatement and key identity

The array receives additions on inclusive ranges. A query asks for the nonnegative GCD of all values in another inclusive range.

Let `d[1] = a[1]` and `d[i] = a[i] - a[i-1]` for `i > 1`. For any `l <= r`,

```
gcd(a[l], a[l+1], ..., a[r])
= gcd(a[l], d[l+1], d[l+2], ..., d[r]).
```

Subtracting `a[l]` from later values does not change their common divisors, and repeated differences give the expression on the right. This identity is the bridge between the range update and the range query.

5.2 What each structure stores

Adding `x` to `[l,r]` changes only two entries of the difference array:

```
d[l]     += x
d[r + 1] -= x       if r < n
```

Use two structures over the same `d`:

- a Fenwick tree stores difference sums, so its prefix sum through `l` reconstructs the current `a[l]`;
- a GCD segment tree stores the differences, so it returns the GCD of `d[l+1..r]` after point changes.

For a query `[l,r]`, combine those two values with `gcd`. The empty difference range when `l = r` contributes zero, which is the identity for GCD.

5.3 Correctness proof

The Fenwick tree and GCD tree initially contain exactly the difference array. A range addition changes its two boundary entries exactly as described, and both structures receive those same point changes. Thus they continue to represent the current array's differences.

For a query `[l,r]`, the Fenwick prefix sum reconstructs the current `a[l]`, and the segment tree returns the GCD of exactly `d[l+1..r]`. By the key identity, the GCD of these quantities equals the GCD of every current array value in `[l,r]`. Taking absolute values makes the printed GCD nonnegative. □

5.4 Complexity and tests

Building both structures costs `O(n)`, or `O(n log n)` with individual Fenwick insertions. Every update performs at most two point changes in each structure, and every query performs one prefix sum and one range GCD, so each operation costs `O(log n)`. Memory is `O(n)`.

Test singleton ranges, updates ending at `n`, negative values, an all-zero range, and repeated additions to the same boundaries. A small literal array with direct range updates and direct GCD queries is an independent oracle.

5.5 Full C++ solution

```cpp
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

struct Fenwick {
    int n;
    vector<ll> bit;

    explicit Fenwick(int n) : n(n), bit(n + 1, 0) {}

    void add(int index, ll delta) {
        for (; index <= n; index += index & -index)
            bit[index] += delta;
    }

    ll prefix_sum(int index) const {
        ll result = 0;
        for (; index > 0; index -= index & -index)
            result += bit[index];
        return result;
    }
};

struct GcdTree {
    int size = 1;
    vector<ll> tree;

    explicit GcdTree(const vector<ll>& values) {
        while (size < static_cast<int>(values.size()))
            size *= 2;
        tree.assign(2 * size, 0);
        for (int i = 0; i < static_cast<int>(values.size()); ++i)
            tree[size + i] = values[i];
        for (int node = size - 1; node > 0; --node)
            pull(node);
    }
```

```
    void pull(int node) {
        tree[node] = gcd(tree[2 * node], tree[2 * node + 1]);
    }

    void add(int position, ll delta) {
        int node = size + position;
        tree[node] += delta;
        for (node /= 2; node > 0; node /= 2)
            pull(node);
    }

    ll query(int left, int right) const {
        ll answer = 0;
        for (left += size, right += size;
             left < right;
             left /= 2, right /= 2) {
            if (left & 1)
                answer = gcd(answer, tree[left++]);
            if (right & 1)
                answer = gcd(answer, tree[--right]);
        }
        return answer;
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, q;
    cin >> n >> q;

    vector<ll> difference(n);
    Fenwick values(n);
    ll previous = 0;
    for (int i = 0; i < n; ++i) {
        ll current;
        cin >> current;
        difference[i] = current - previous;
        previous = current;
        values.add(i + 1, difference[i]);
    }

    GcdTree differences(difference);

    while (q--) {
        int type, left, right;
        cin >> type >> left >> right;
        --left;
```

```
        if (type == 1) {
            ll delta;
            cin >> delta;

            differences.add(left, delta);
            values.add(left + 1, delta);
            if (right < n) {
                differences.add(right, -delta);
                values.add(right + 1, -delta);
            }
        } else {
            ll first_value = values.prefix_sum(left + 1);
            ll inside_differences = differences.query(left + 1, right);
            cout << gcd(abs(first_value), abs(inside_differences)) << '\n';
        }
    }
}
```

5.6 Full Python solution

```
import math
import sys


class Fenwick:
    def __init__(self, n):
        self.n = n
        self.bit = [0] * (n + 1)

    def add(self, index, delta):
        while index <= self.n:
            self.bit[index] += delta
            index += index & -index

    def prefix_sum(self, index):
        result = 0
        while index > 0:
            result += self.bit[index]
            index -= index & -index
        return result


class GcdTree:
    def __init__(self, values):
        self.size = 1
        while self.size < len(values):
            self.size *= 2
        self.tree = [0] * (2 * self.size)
```

```
        self.tree[self.size:self.size + len(values)] = values
        for node in range(self.size - 1, 0, -1):
            self._pull(node)

    def _pull(self, node):
        self.tree[node] = math.gcd(
            self.tree[2 * node],
            self.tree[2 * node + 1],
        )

    def add(self, position, delta):
        node = self.size + position
        self.tree[node] += delta
        node //= 2
        while node:
            self._pull(node)
            node //= 2

    def query(self, left, right):
        answer = 0
        left += self.size
        right += self.size
        while left < right:
            if left & 1:
                answer = math.gcd(answer, self.tree[left])
                left += 1
            if right & 1:
                right -= 1
                answer = math.gcd(answer, self.tree[right])
            left //= 2
            right //= 2
        return answer


def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    n, q = data[0], data[1]
    array = data[2:2 + n]
    pointer = 2 + n

    difference = [array[0]]
    difference.extend(array[i] - array[i - 1] for i in range(1, n))

    values = Fenwick(n)
    for index, delta in enumerate(difference, 1):
        values.add(index, delta)
    differences = GcdTree(difference)

    output = []
    for _ in range(q):
        operation = data[pointer]
```

```
        left = data[pointer + 1] - 1
        right = data[pointer + 2]
        pointer += 3

        if operation == 1:
            delta = data[pointer]
            pointer += 1
            differences.add(left, delta)
            values.add(left + 1, delta)
            if right < n:
                differences.add(right, -delta)
                values.add(right + 1, -delta)
        else:
            first_value = values.prefix_sum(left + 1)
            inside = differences.query(left + 1, right)
            output.append(str(math.gcd(abs(first_value), abs(inside))))

    print('\n'.join(output))


if __name__ == '__main__':
    main()
```

6 D. Salary Queries

6.1 Restatement and constraint implications

There are `n` employees with current integer salaries. An operation `! k x` changes employee `k`'s salary to `x`. A query `? a b` asks how many current salaries lie in the inclusive *value* interval `[a,b]`. There are up to `200,000` employees and operations, and salaries can be negative or as large as `10^9` in magnitude.

The interval is about salary values, not employee indices. Scanning all employees per query is quadratic. A query needs only the number of occurrences of each ordered salary value, which is exactly what a Fenwick tree can store after coordinate compression.

6.2 Coordinate collection and trace

Read and retain all operations first. Collect every initial salary and every new `x` in an update, sort the distinct values, and give each one a one-based rank. This preserves order: for collected salaries `u` and `v`, `u < v` if and only if `rank(u) < rank(v)`.

For initial salaries `[10,20,30,40,50]`, query `? 15 45` has four salaries at most 45 and one salary below 15, so its answer is `4-1=3`. After `! 2 35`, remove one count at rank 20 and add one at rank 35. Then `? 30 35` returns two. The bounds 15 and 45 need not be coordinates: binary search locates where they fall among actual salary values.

6.3 Algorithm

```
collect initial salaries and every update value
sort and deduplicate the collected values
insert all current salary ranks into a Fenwick frequency tree

for each operation:
    if ! k x:
        remove the rank of current[k]
        set current[k] = x and add its rank
    else ? a b:
        right = number of coordinate values <= b
        left = number of coordinate values < a
        print BIT.prefix(right) - BIT.prefix(left)
```

For the right boundary, use C++ `upper_bound` or Python `bisect_right`. For the left boundary, use C++ `lower_bound` or Python `bisect_left`.

6.4 Correctness proof

The Fenwick invariant is: the frequency at each rank equals the number of employees currently earning that corresponding salary. Initialization adds each employee once. An update removes exactly its old salary and adds exactly its new one, so the invariant remains true.

For `[a,b]`, `right` covers exactly salaries at most `b`, while `left` covers exactly salaries below `a`. The two Fenwick prefix sums therefore count those two employee sets under the invariant. Their difference counts exactly the employees with `a <= salary <= b`, so every query answer is correct. QED.

6.5 Complexity, implementation notes, and tests

Sorting coordinates costs `O((n+q) log(n+q))`. Each update/query uses a constant number of binary searches and Fenwick operations, hence `O(log(n+q))`; memory is `O(n+q)`. Use `long long` for C++ salary values.

Test an update below every old salary, equal salaries, query endpoints absent from coordinates, and an empty value range. Common mistakes are using a lower bound for the inclusive upper endpoint, or adding the new salary without first removing the old one.

6.6 Full C++ solution

```cpp
#include <bits/stdc++.h>
using namespace std;

struct Fenwick {
    vector<int> bit;
    explicit Fenwick(int n) : bit(n + 1, 0) {}

    void add(int index, int delta) {
        int size = static_cast<int>(bit.size());
        for (; index < size; index += index & -index) {
            bit[index] += delta;
        }
    }

    int sum(int index) const {
        int result = 0;
        for (; index > 0; index -= index & -index)
            result += bit[index];
        return result;
    }
};

struct Operation {
    char type;
    int first;
```

```
    long long second;
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, q;
    if (!(cin >> n >> q)) return 0;
    vector<long long> salary(n + 1), coords;
    for (int i = 1; i <= n; i++) {
        cin >> salary[i];
        coords.push_back(salary[i]);
    }

    vector<Operation> operations;
    operations.reserve(q);
    for (int i = 0; i < q; i++) {
        char type;
        long long first, second;
        cin >> type >> first >> second;
        operations.push_back({type, (int)first, second});
        if (type == '!') coords.push_back(second);
    }

    sort(coords.begin(), coords.end());
    auto unique_end = unique(coords.begin(), coords.end());
    coords.erase(unique_end, coords.end());
    auto rank_of = [&](long long value) {
        auto iterator = lower_bound(
            coords.begin(), coords.end(), value
        );
        return int(iterator - coords.begin()) + 1;
    };

    Fenwick bit((int)coords.size());
    for (int i = 1; i <= n; i++) bit.add(rank_of(salary[i]), 1);

    for (const Operation& operation : operations) {
        if (operation.type == '!') {
            int employee = operation.first;
            bit.add(rank_of(salary[employee]), -1);
            salary[employee] = operation.second;
            bit.add(rank_of(salary[employee]), 1);
        } else {
            long long low = operation.first;
            long long high = operation.second;
            auto low_iterator = lower_bound(
                coords.begin(), coords.end(), low
            );
            auto high_iterator = upper_bound(
```

```
                coords.begin(), coords.end(), high
            );
            int before_low = int(low_iterator - coords.begin());
            int through_high = int(high_iterator - coords.begin());
            int answer = (
                bit.sum(through_high) - bit.sum(before_low)
            );
            cout << answer << '\n';
        }
    }
    return 0;
}
```

6.7 Full Python solution

```
import bisect
import sys

class Fenwick:

    def __init__(self, size: int) -> None:
        self.bit = [0] * (size + 1)

    def add(self, index: int, delta: int) -> None:
        while index < len(self.bit):
            self.bit[index] += delta
            index += index & -index

    def prefix_sum(self, index: int) -> int:
        total = 0
        while index:
            total += self.bit[index]
            index -= index & -index
        return total

def main() -> None:
    tokens = sys.stdin.buffer.read().split()
    if not tokens:
        return
    pointer = 0
    n = int(tokens[pointer])
    q = int(tokens[pointer + 1])
    pointer += 2
    initial = tokens[pointer:pointer + n]
    salary = [0] + [int(value) for value in initial]
    pointer += n
    operations: list[tuple[bytes, int, int]] = []
    coords = salary[1:]
```

```
    for _ in range(q):
        kind = tokens[pointer]
        first = int(tokens[pointer + 1])
        second = int(tokens[pointer + 2])
        pointer += 3
        operations.append((kind, first, second))
        if kind == b'!':
            coords.append(second)
    coords = sorted(set(coords))
    bit = Fenwick(len(coords))

    def rank_of(value: int) -> int:
        return bisect.bisect_left(coords, value) + 1
    for value in salary[1:]:
        bit.add(rank_of(value), 1)
    output: list[str] = []
    for kind, first, second in operations:
        if kind == b'!':
            employee, new_salary = (first, second)
            bit.add(rank_of(salary[employee]), -1)
            salary[employee] = new_salary
            bit.add(rank_of(new_salary), 1)
        else:
            low, high = (first, second)
            before_low = bisect.bisect_left(coords, low)
            through_high = bisect.bisect_right(coords, high)
            answer = (
                bit.prefix_sum(through_high)
                - bit.prefix_sum(before_low)
            )
            output.append(str(answer))
    print('\n'.join(output))
if __name__ == '__main__':
    main()
```

7 Testing discussion

The small D cases keep a literal salary list and answer queries by scanning it; they never invoke a reference solution. The scale case combines 100,000 updates with 100,000 range-count queries, so a linear scan cannot pass. The expected outputs are shared by the C++ and Python runners.

8 Detailed upsolve workshop

8.1 A. Hotel Queries

Constraints and trap. `n,q` are up to about `2e5`. A linear scan for each group is too slow; a plain maximum alone finds feasibility but not the first index. Store the maximum remaining capacity in each segment-tree node.

Trace and pseudocode. With capacities `[3,1,4,1,5]`, a request of four descends root -> left/right according to whether the left subtree maximum is at least four, reaches index three, and changes it to zero.

```
if root.max < x: output 0
otherwise descend: go left if left.max >= x, else right
at leaf i: subtract x
recompute maxima on the path to the root
output i
```

Proof and complexity. Every node is the maximum of exactly its leaves. A feasible left child contains an earlier valid hotel; an infeasible left child contains none, so the descent returns the first valid leaf. Each query changes and rebuilds one root-to-leaf path: `O(log n)` time, `O(n)` memory. Use `long long` capacities. Test no feasible hotel, the first/last feasible hotel, and repeated requests to one hotel.

8.2 B. List Removals

Constraints and trap. Removing an element from an array/vector shifts all following values and becomes quadratic. Instead, original positions are fixed; a Fenwick tree stores one for alive positions and zero for removed positions.

Trace and pseudocode. For values `[10,20,30,40,50]`, rank two resolves to original index two, outputs 20, then adds -1. With counts `[1,0,1,1,1]`, rank three resolves to original index four, outputting 40.

```
initialize BIT with one at every index
for rank k:
    i = first index whose BIT prefix sum >= k
    output value[i]; BIT.add(i,-1)
```

Proof and complexity. The BIT prefix sum is the number of alive original positions at or before its index. Therefore its first prefix reaching `k` is the k-th current item. Fenwick binary lifting gives `O(log n)` per removal and `O(n)` memory. The search uses one-based indices; test rank one, last rank, and duplicate values. A small mutable-list simulation is the stress oracle.

8.3 C. Range-add/GCD upsolve

Primary diagnosis. A lazy GCD tree is awkward because adding the same number does not determine a segment's new GCD from its old GCD. Differences make the update sparse: only `d[l]` and possibly `d[r+1]` change.

```
update [l,r] by x:
    point-add x at d[l]
    if r < n: point-add -x at d[r+1]

query [l,r]:
    first = prefix_sum(d, l)
    rest = range_gcd(d, l+1, r)
    answer = gcd(abs(first), abs(rest))
```

The prefix sum reconstructs the first value. The remaining differences are enough because `gcd(x,y) = gcd(x,y-x)`. Every operation changes or reads only `O(log n)` nodes in each structure.

Test `l=r`, `r=n`, zeros, negative updates, and ranges whose GCD increases after an update. The random oracle modifies a literal array, then scans the queried range with `gcd`.

9 What to carry forward

Choose a representation that makes the update sparse, then store the summaries the query actually needs: subtree maximum for first-fit, alive prefix count for dynamic rank, and both prefix sums and range GCDs for a difference array. The invariant makes both the data structure and its proof mechanical.

The fixed samples cover visible behavior. The random tests generate many small operation sequences and compute expected outputs with direct brute-force models. The same expected files are used against both the C++ and Python submissions.

10 E. Functional Walk Minimum

10.1 Restatement and constraint recognition

Every vertex has exactly one outgoing edge. A query starts at `v`, follows exactly `k` edges, and asks for both the final vertex and the minimum value among the start and all reached vertices.

Simulating a query takes `O(k)`, but `k` may be as large as `10^18`. The functional graph never branches during a walk, so binary lifting can precompute the effect of blocks of `1,2,4,8,...` steps. Alongside the destination of a block, store an aggregate—in this case, a minimum.

10.2 Table meaning

For vertex `v` and level `j`, define:

```
up[v][j] = vertex reached after exactly 2^j edges from v
mn[v][j] = minimum value among the 2^j reached vertices
           (the starting value at v is not included)
```

Excluding the block's starting value avoids double-counting concerns when concatenating blocks. The query initializes its answer with `value[v]`, so the original start is still included exactly as required.

For level zero:

```
up[v][0] = next[v]
mn[v][0] = value[next[v]]
```

Two adjacent blocks of length `2^(j-1)` give:

```
middle   = up[v][j-1]
up[v][j] = up[middle][j-1]
mn[v][j] = min(mn[v][j-1], mn[middle][j-1])
```

10.3 Answering a query

Write `k` in binary. Starting with `answer = value[v]`, inspect its set bits from low to high. For bit `j`, merge `mn[v][j]` into the answer and replace `v` by `up[v][j]`. The selected blocks concatenate to exactly `k` edges. Sixty-one levels cover every `k <= 10^18`.

10.4 Correctness proof

By induction on `j`, `up[v][j]` reaches the endpoint of exactly `2^j` edges and `mn[v][j]` is the minimum over precisely the vertices reached by those edges. The base case is the single outgoing edge. For the induction step, the first stored half starts at `v`; the second starts at its endpoint `middle`. Their concatenation is the required full block, and taking the minimum of their aggregates is exact.

During a query, after processing some bits, `v` is the endpoint of the corresponding concatenated blocks and `answer` is the minimum over the original start and all vertices reached so far. Processing a set bit appends the correct next block by the table lemma and merges exactly its values. After all bits, the blocks total `k`, proving both reported values correct.

10.5 Complexity and implementation notes

With `L = 61`, preprocessing costs `O(nL)` time and memory, and each query costs `O(L)`. In asymptotic terms this is `O((n+q) log K)` time and `O(n log K)` memory.

The Python reference stores tables in packed `array` objects rather than lists of Python integers; at `n = 200000`, ordinary nested lists would consume far more memory. Be careful that `k = 0` must return the original vertex and its value, and that the block minimum includes destinations but not its start.

10.6 Tests that matter

```
k = 0                          -> unchanged vertex and starting value
k = 1                          -> one table base block
k with several set bits        -> block concatenation
A self-loop                     -> repeated endpoint
A tail entering a cycle        -> works without cycle detection
Minimum occurs at the start, in the tail, and inside the cycle
k = 10^18                      -> verifies the number of levels
```

For small graphs and small `k`, direct step-by-step simulation is an independent oracle.

10.7 Full C++ solution

```cpp
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
```

```
    int n, q;
    cin >> n >> q;

    constexpr int LEVELS = 61;
    vector<array<int, LEVELS>> up(n);
    vector<array<ll, LEVELS>> minimum(n);

    for (int vertex = 0; vertex < n; ++vertex) {
        cin >> up[vertex][0];
        --up[vertex][0];
    }

    vector<ll> value(n);
    for (ll& current : value)
        cin >> current;

    for (int vertex = 0; vertex < n; ++vertex)
        minimum[vertex][0] = value[up[vertex][0]];

    for (int level = 1; level < LEVELS; ++level) {
        for (int vertex = 0; vertex < n; ++vertex) {
            int middle = up[vertex][level - 1];
            up[vertex][level] = up[middle][level - 1];
            minimum[vertex][level] = min(
                minimum[vertex][level - 1],
                minimum[middle][level - 1]
            );
        }
    }

    while (q--) {
        int vertex;
        unsigned long long steps;
        cin >> vertex >> steps;
        --vertex;

        ll answer = value[vertex];
        for (int level = 0; level < LEVELS; ++level) {
            if ((steps >> level) & 1ULL) {
                answer = min(answer, minimum[vertex][level]);
                vertex = up[vertex][level];
            }
        }

        cout << vertex + 1 << ' ' << answer << '\n';
    }
}
```

10.8 Full Python solution

The Python table build iterates directly over the previous jump endpoints. For queries it visits only set bits, in increasing order, and compares the block minimum directly. These reduce overhead without changing the jump invariants or the `O(log k)` query bound.

```
import sys
from array import array

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    n, q = data[:2]
    next_vertex = array('i', (x - 1 for x in data[2:2 + n]))
    value = data[2 + n:2 + 2 * n]
    index = 2 + 2 * n
    levels = 61
    up = [next_vertex]
    first_minimum = (value[next_vertex[v]] for v in range(n))
    minimum = [array('q', first_minimum)]
    for level in range(1, levels):
        previous_up = up[level - 1]
        previous_minimum = minimum[level - 1]
        up.append(array('i', [previous_up[p] for p in previous_up]))
        destinations = (previous_minimum[p] for p in previous_up)
        minimum.append(array('q', [
            a if a < b else b
            for a, b in zip(previous_minimum, destinations)
        ]))
    output = []
    for _ in range(q):
        vertex = data[index] - 1
        steps = data[index + 1]
        index += 2
        answer = value[vertex]
        while steps:
            # Consume just the set bits, in increasing jump order.
            bit = (steps & -steps).bit_length() - 1
            block_minimum = minimum[bit][vertex]
            if block_minimum < answer:
                answer = block_minimum
            vertex = up[bit][vertex]
            steps &= steps - 1
        output.append(f'{vertex + 1} {answer}')
    sys.stdout.write('\n'.join(output))
if __name__ == '__main__':
    main()
```

11 F. Mutable Priority Queue

11.1 Why an ordinary heap is insufficient

The collection supports additions, priority changes, and removal of the task with greatest priority. A max-heap finds the best task, but changing the priority of an arbitrary task would require locating its old entry inside the heap. A standard binary heap does not support that search efficiently.

Keep two representations:

```
current[id] = authoritative current priority
heap entry   = one historical (priority,id) version
```

On an add or update, write the new value to `current` and push a new heap entry. Do not search for the old one.

11.2 Cleaning the top

Before every pop operation, repeatedly inspect the heap top. Its entry is valid exactly when the identifier is present in `current` and the stored priority equals `current[id]`. Otherwise it is stale and can be discarded.

Order heap entries by greatest priority, then smallest identifier. C++ can store `(priority,-id)` in its default max-heap. Python can store `(-priority,id)` in its default min-heap.

After cleaning, an empty heap means the collection is empty. Otherwise print the top identifier, remove that entry, and erase the identifier from `current`. Erasing it also makes any older copies of that identifier stale.

11.3 Correctness proof

The dictionary invariant is that it contains exactly the active tasks and their current priorities. Adds, updates, and successful pops maintain this invariant directly.

Every active task has a heap entry matching its dictionary value: its most recent add or update inserted one, and only popping that task removes the dictionary entry. Cleaning removes only entries that disagree with the dictionary, so it never removes the current version of an active task. Therefore, after cleaning, the heap top is the greatest-priority active task; the heap's tie order selects the smallest identifier. The printed answer is exactly the required task. □

11.4 Complexity and tests

Each add/update inserts one entry in `O(log q)`. An individual pop may discard several stale entries, but every entry is inserted once and removed at most once. Across all operations, cleanup therefore costs `O(q log q)`, giving amortized `O(log q)` time per operation and `O(q)` memory.

Test equal priorities, repeated updates of one task, changing a task from the best to the worst and back, popping an empty collection, and reusing an identifier after removal.

11.5 Full C++ solution

```cpp
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int q;
    cin >> q;

    priority_queue<pair<ll, int>> heap;
    unordered_map<int, ll> current;

    while (q--) {
        char operation;
        cin >> operation;

        if (operation == 'A' || operation == 'U') {
            int id;
            ll priority;
            cin >> id >> priority;
            current[id] = priority;
            heap.push({priority, -id});
            continue;
        }

        while (!heap.empty()) {
            auto [priority, negative_id] = heap.top();
            int id = -negative_id;
            auto found = current.find(id);
            if (found != current.end()
                && found->second == priority) {
                break;
            }
            heap.pop();
        }
```

```
        if (heap.empty()) {
            cout << -1 << '\n';
        } else {
            int id = -heap.top().second;
            heap.pop();
            current.erase(id);
            cout << id << '\n';
        }
    }
}
```

11.6 Full Python solution

```
import heapq
import sys


def main():
    lines = sys.stdin.buffer.readlines()
    current = {}
    heap = []
    output = []

    for line in lines[1:]:
        fields = line.split()
        operation = fields[0]

        if operation in (b'A', b'U'):
            identifier = int(fields[1])
            priority = int(fields[2])
            current[identifier] = priority
            heapq.heappush(heap, (-priority, identifier))
            continue

        while heap:
            negative_priority, identifier = heap[0]
            priority = -negative_priority
            if current.get(identifier) == priority:
                break
            heapq.heappop(heap)

        if not heap:
            output.append("-1")
        else:
            _, identifier = heapq.heappop(heap)
            del current[identifier]
            output.append(str(identifier))
```

```
    sys.stdout.write('\n'.join(output))


if __name__ == '__main__':
    main()
```

12 G. Streaming Room Count

12.1 Model and algorithm

Because meetings arrive by starting time, keep the ending times of precisely the meetings active at the next start in a min-heap. Before inserting `[s,e)`, remove every ending time at most `s`. Insert `e` and update the largest heap size ever seen.

12.2 Why it works

After the removals, every heap entry overlaps the new meeting, and every previous meeting not in the heap has ended. Thus the heap size after insertion is exactly the simultaneous load at `s`. The minimum room count is the maximum simultaneous load over all starts, which the running maximum records.

Time is `O(n log n)` and memory is `O(n)`. The important boundary is that an end equal to a start is reusable because intervals are half-open.

12.3 Full solutions

```
#include <bits/stdc++.h>
using namespace std;
int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);
  int n;
  if (!(cin >> n))
    return 0;
  priority_queue<
      long long,
      vector<long long>,
      greater<long long>
  > ends;
  int answer = 0;
  for (int i = 0; i < n; i++) {
    long long s, e;
    cin >> s >> e;
    while (!ends.empty() && ends.top() <= s)
      ends.pop();
    ends.push(e);
    answer = max(answer, (int)ends.size());
    if (i)
```

```
      cout << ' ';
    cout << answer;
  }
  cout << '\n';
}
```

```
import heapq, sys

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n = data[0]
    ends = []
    answer = 0
    out = []
    j = 1
    for _ in range(n):
        s, e = data[j:j + 2]
        j += 2
        while ends and ends[0] <= s:
            heapq.heappop(ends)
        heapq.heappush(ends, e)
        answer = max(answer, len(ends))
        out.append(str(answer))
    print(*out)
if __name__ == '__main__':
    main()
```

13 H. Toggle K-th Active

13.1 Model and algorithm

Store `1` for an active position and `0` otherwise in a Fenwick tree. Toggling is a point addition. A `K k` query asks for the smallest index whose prefix sum is at least `k`; find it by Fenwick binary lifting after checking that the total count reaches `k`.

13.2 Why it works

Every Fenwick prefix is exactly the number of active positions in that prefix. Therefore the first prefix reaching `k` ends at the k-th active position. Binary lifting maintains the largest prefix whose count is still below `k`, so the following index is the answer.

Each operation costs `O(log n)` and memory is `O(n)`. Test repeated toggles, `k=1`, `k=total`, and an invalid `k`.

13.3 Full solutions

```cpp
#include <bits/stdc++.h>
using namespace std;
struct BIT {
  int n;
  vector<int> t;
  BIT(int n) : n(n), t(n + 1) {}
  void add(int i, int v) {
    for (; i <= n; i += i & -i)
      t[i] += v;
  }
  int sum(int i) {
    int s = 0;
    for (; i; i -= i & -i)
      s += t[i];
    return s;
  }
  int kth(int k) {
    int p = 0;
    for (int d = bit_floor((unsigned)n); d; d >>= 1)
      if (p + d <= n && t[p + d] < k)
        k -= t[p += d];
    return p + 1;
  }
```

```
};
int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);
  int n, q;
  if (!(cin >> n >> q))
    return 0;
  BIT b(n);
  vector<char> on(n + 1);
  while (q--) {
    char c;
    int x;
    cin >> c >> x;
    if (c == 'T') {
      b.add(x, on[x] ? -1 : 1);
      on[x] ^= 1;
    } else
      cout << (b.sum(n) < x ? -1 : b.kth(x)) << '\n';
  }
}
```

```
import sys

def main():
    data = sys.stdin.buffer.read().split()
    if not data:
        return
    n, q = map(int, data[:2])
    bit = [0] * (n + 1)
    on = [False] * (n + 1)
    out = []
    j = 2

    def add(i, v):
        while i <= n:
            bit[i] += v
            i += i & -i

    def total():
        i = n
        s = 0
        while i:
            s += bit[i]
            i -= i & -i
        return s

    def kth(k):
        p = 0
        d = 1 << n.bit_length() - 1
        while d:
            np = p + d
```

```
            if np <= n and bit[np] < k:
                p = np
                k -= bit[np]
            d //= 2
        return p + 1
    for _ in range(q):
        c = data[j]
        x = int(data[j + 1])
        j += 2
        if c == b'T':
            add(x, -1 if on[x] else 1)
            on[x] = not on[x]
        else:
            out.append(str(kth(x) if total() >= x else -1))
    print('\n'.join(out))
if __name__ == '__main__':
    main()
```

14 I. Dynamic Brackets

14.1 Reduction and node invariant

Encode `(` as `+1` and `)` as `-1`. A substring is balanced precisely when its total is zero and none of its prefix sums is negative. A total of zero alone is insufficient: `)(` is a counterexample.

Store `(sum, low)` in each segment-tree node. `low` is the minimum prefix sum, including the empty prefix. For adjacent segments `L` and `R`:

```
sum = L.sum + R.sum
low = min(L.low, L.sum + R.low)
```

A leaf stores `(1,0)` or `(-1,-1)`. The empty segment stores `(0,0)`. Flipping a bracket changes one leaf and recomputes its ancestors.

14.2 Ordered interval queries

Concatenation is associative but not commutative. An iterative query must append left-side blocks to a left accumulator and prepend right-side blocks to a right accumulator, then concatenate left and right. Reversing any of these orders changes which prefixes are examined.

For a query, print `YES` exactly when `sum=0` and `low>=0`. The sample's first flip makes the whole string start with `)`, while an interior `()` query remains balanced. All summaries are relative to the queried substring; preceding brackets do not help it.

14.3 Correctness and complexity

Every prefix of a concatenation either ends inside `L`, with minimum `L.low`, or includes all of `L` and a prefix of `R`, with minimum `L.sum+R.low`. Thus the merge preserves its invariant. Induction over tree nodes, and the ordered query merge, recover the exact total and minimum prefix. These are precisely the two conditions for balanced parentheses.

Build in `O(n)` time. Each flip or query takes `O(log n)` time; memory is `O(n)`. Test single characters, `)(`, nested sequences, repeated flips, substrings starting in the middle, and a zero-total full string with a negative first prefix. The dedicated tests include long queried ranges and that last failure case.

14.4 Full solutions

```cpp
#include <bits/stdc++.h>
using namespace std;

struct Node { int sum = 0, low = 0; };
Node merge_nodes(Node a, Node b) {
    return {a.sum+b.sum, min(a.low, a.sum+b.low)};
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    if (!(cin >> n >> q)) return 0;
    string s; cin >> s;
    int size = 1;
    while (size < n) size *= 2;
    vector<Node> tree(2*size);
    for (int i = 0; i < n; i++) {
        int x = s[i] == '(' ? 1 : -1;
        tree[size+i] = {x, min(0, x)};
    }
    for (int v = size-1; v > 0; v--)
        tree[v] = merge_nodes(tree[2*v], tree[2*v+1]);
    while (q--) {
        char op; cin >> op;
        if (op == 'T') {
            int i; cin >> i;
            int v = size+i-1, x = -tree[v].sum;
            tree[v] = {x, min(0, x)};
            for (v /= 2; v > 0; v /= 2)
                tree[v] = merge_nodes(tree[2*v], tree[2*v+1]);
        } else {
            int l, r; cin >> l >> r;
            l += size-1; r += size;
            Node left, right;
            while (l < r) {
                if (l & 1) left = merge_nodes(left, tree[l++]);
                if (r & 1) right = merge_nodes(tree[--r], right);
                l /= 2; r /= 2;
            }
            Node answer = merge_nodes(left, right);
            cout << (answer.sum == 0 && answer.low >= 0 ? "YES" : "NO")
                 << '\n';
        }
    }
}
```

```python
import sys
```

```
def main():
    data = sys.stdin.buffer.read().split()
    n, q = map(int, data[:2])
    s = bytearray(data[2])
    size = 1 << (n - 1).bit_length()
    total = [0] * (2 * size)
    low = [0] * (2 * size)
    for i, c in enumerate(s):
        total[size + i] = 1 if c == 40 else -1
        low[size + i] = min(0, total[size + i])
    for v in range(size - 1, 0, -1):
        total[v] = total[2*v] + total[2*v+1]
        low[v] = min(low[2*v], total[2*v] + low[2*v+1])
    out = []
    pos = 3
    for _ in range(q):
        op = data[pos]; pos += 1
        if op == b'T':
            i = int(data[pos]) - 1; pos += 1
            v = size + i
            total[v] = -total[v]; low[v] = min(0, total[v])
            v //= 2
            while v:
                total[v] = total[2*v] + total[2*v+1]
                low[v] = min(low[2*v], total[2*v] + low[2*v+1])
                v //= 2
        else:
            l, r = int(data[pos]) - 1 + size, int(data[pos+1]) + size
            pos += 2
            ls = lm = rs = rm = 0
            while l < r:
                if l & 1:
                    lm = min(lm, ls + low[l]); ls += total[l]; l += 1
                if r & 1:
                    r -= 1
                    rm = min(low[r], total[r] + rm); rs += total[r]
                l //= 2; r //= 2
            out.append('YES' if ls + rs == 0 and min(lm, ls + rm) >= 0 else 'NO')
    print('\n'.join(out))


if __name__ == '__main__':
    main()
```

15 J. Sliding Median Cost

15.1 Reduction

The minimizing `x` is any median. Coordinate-compress all values and maintain two Fenwick trees for the current window: one stores counts and the other stores sums. Find the lower median `m` by order statistics. If `(cL,sL)` are the count and sum at most `m`, and `(cR,sR)` are those above it, the cost is

```
m*cL - sL + sR - m*cR.
```

15.2 Correctness and complexity

Moving `x` toward a median cannot increase the number of values moved away from compared with values moved toward; hence a median minimizes absolute deviation. The formula separately sums `m-a[i]` on the left and `a[i]-m` on the right. Each insertion, deletion, and median search is `O(log n)`, for `O(n log n)` total time and `O(n)` memory.

15.3 Full solutions

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
struct BIT {
  int n;
  vector<ll> t;
  BIT(int n) : n(n), t(n + 1) {}
  void add(int i, ll v) {
    for (; i <= n; i += i & -i)
      t[i] += v;
  }
  ll sum(int i) {
    ll s = 0;
    for (; i; i -= i & -i)
      s += t[i];
    return s;
  }
  int kth(ll k) {
    int p = 0;
    for (int d = bit_floor((unsigned)n); d; d >>= 1)
      if (p + d <= n && t[p + d] < k)
```

```
        k -= t[p += d];
    return p + 1;
  }
};
int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);
  int n, k;
  if (!(cin >> n >> k))
    return 0;
  vector<ll> a(n), v;
  for (ll &x : a)
    cin >> x;
  v = a;
  sort(v.begin(), v.end());
  v.erase(unique(v.begin(), v.end()), v.end());
  BIT cnt(v.size()), sum(v.size());
  auto add = [&](ll x, int d) {
    int p = lower_bound(v.begin(), v.end(), x) - v.begin() + 1;
    cnt.add(p, d);
    sum.add(p, d * x);
  };
  for (int i = 0; i < k; i++)
    add(a[i], 1);
  for (int l = 0;; l++) {
    int p = cnt.kth((k + 1) / 2);
    ll m = v[p - 1];
    ll cl = cnt.sum(p);
    ll sl = sum.sum(p);
    ll ct = cnt.sum(v.size());
    ll st = sum.sum(v.size());
    ll ans = m * cl - sl + (st - sl) - m * (ct - cl);
    if (l)
      cout << ' ';
    cout << ans;
    if (l + k == n)
      break;
    add(a[l], -1);
    add(a[l + k], 1);
  }
  cout << '\n';
}
```

```
import bisect, sys

def main():
    d = list(map(int, sys.stdin.buffer.read().split()))
    if not d:
        return
    n, k = d[:2]
    a = d[2:]
```

```
    v = sorted(set(a))
    z = len(v)
    bc = [0] * (z + 1)
    bs = [0] * (z + 1)

    def add(bit, i, x):
        while i <= z:
            bit[i] += x
            i += i & -i

    def pref(bit, i):
        s = 0
        while i:
            s += bit[i]
            i -= i & -i
        return s

    def change(x, d):
        p = bisect.bisect_left(v, x) + 1
        add(bc, p, d)
        add(bs, p, d * x)

    def kth(kth):
        p = 0
        step = 1 << z.bit_length() - 1
        while step:
            np = p + step
            if np <= z and bc[np] < kth:
                p = np
                kth -= bc[np]
            step //= 2
        return p + 1
    for x in a[:k]:
        change(x, 1)
    out = []
    for l in range(n - k + 1):
        p = kth((k + 1) // 2)
        m = v[p - 1]
        cl = pref(bc, p)
        sl = pref(bs, p)
        ct = pref(bc, z)
        st = pref(bs, z)
        out.append(str(m * cl - sl + st - sl - m * (ct - cl)))
        if l + k < n:
            change(a[l], -1)
            change(a[l + k], 1)
    print(*out)
if __name__ == '__main__':
    main()
```

16 K. Budget Prefix Search

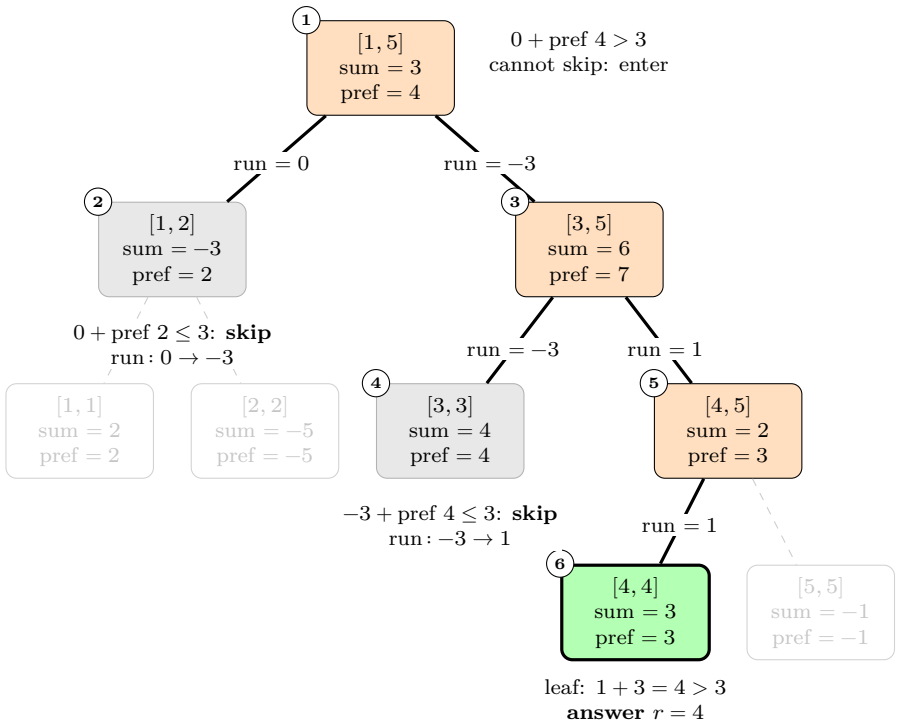
16.1 Segment information and guided descent

Signed values destroy monotonicity: a running sum may cross the budget, fall back, and cross again. Store for every segment its total sum and its maximum nonempty prefix sum. The merge is

```
sum  = left.sum + right.sum
pref = max(left.pref, left.sum + right.pref).
```

During a query, maintain the sum of the already skipped part of `[l,n)`. A whole segment can be skipped exactly when `running + segment.pref <= budget`; then add its total to `running`. Otherwise descend left before right. The first leaf that cannot be skipped is the first budget breach.

The sample query `Q 1 3` on `a = [2,-5,4,3,-1]` exercises every case. Orange nodes are entered because their prefix maximum reaches past the budget, grey nodes are skipped whole and only contribute their total to `running`, and the green leaf is the first breach. Badges give the visit order:



	$i=1$	$i=2$	$i=3$	$i=4$	$i=5$
a_i	2	−5	4	3	−1
run	2	−3 ②	1 ④	4	3

The prefix sums from `l = 1` are `2, -3, 1, 4, 3`. The walk dips to `-3` before it first exceeds `b = 3` at `r = 4`, so no binary search over those sums is available. The descent never materialises the whole walk: the only two values `running` ever takes are `-3` after step 2 and `1` after step 4, exactly the prefix sums at the right ends of the skipped segments.

The stored maximum examines every prefix of a segment: it either stays in the left child or contains all of the left child and a prefix of the right. Thus a skipped node contains no breach. Descending left first returns the earliest node that does contain one. Point assignments and searches both cost `O(log n)`, with `O(n)` memory.

16.2 Full solutions

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
struct Node {
  ll sum, pref;
};
struct Seg {
  int n;
  vector<Node> t;
  Seg(vector<ll> &a) : n(a.size()), t(4 * n) { build(1, 0, n, a); }
  void pull(int x) {
    t[x] = {t[2 * x].sum + t[2 * x + 1].sum,
            max(t[2 * x].pref, t[2 * x].sum + t[2 * x + 1].pref)};
  }
  void build(int x, int l, int r, vector<ll> &a) {
    if (r - l == 1) {
      t[x] = {a[l], a[l]};
      return;
    }
    int m = (l + r) / 2;
    build(2 * x, l, m, a);
    build(2 * x + 1, m, r, a);
    pull(x);
  }
  void set_value(int x, int l, int r, int p, ll v) {
    if (r - l == 1) {
      t[x] = {v, v};
      return;
    }
    int m = (l + r) / 2;
    if (p < m)
      set_value(2 * x, l, m, p, v);
    else
      set_value(2 * x + 1, m, r, p, v);
    pull(x);
  }
  int first(int x, int l, int r, int ql, ll budget, ll &sum) {
```

```
    if (r <= ql)
      return -1;
    if (ql <= l && sum + t[x].pref <= budget) {
      sum += t[x].sum;
      return -1;
    }
    if (r - l == 1)
      return l;
    int m = (l + r) / 2, res = first(2 * x, l, m, ql, budget, sum);
    return res < 0 ? first(2 * x + 1, m, r, ql, budget, sum) : res;
  }
};
int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);
  int n, q;
  if (!(cin >> n >> q))
    return 0;
  vector<ll> a(n);
  for (ll &x : a)
    cin >> x;
  Seg s(a);
  while (q--) {
    char c;
    int i;
    ll x;
    cin >> c >> i >> x;
    --i;
    if (c == 'U')
      s.set_value(1, 0, n, i, x);
    else {
      ll sum = 0;
      int answer = s.first(1, 0, n, i, x, sum);
      cout << (answer < 0 ? -1 : answer + 1) << '\n';
    }
  }
}
```

```
import sys

def main():
    d = sys.stdin.buffer.read().split()
    if not d:
        return
    n, q = map(int, d[:2])
    a = list(map(int, d[2:2 + n]))
    total = [0] * (4 * n)
    pref = [0] * (4 * n)

    def pull(x):
        total[x] = total[2 * x] + total[2 * x + 1]
```

```
        pref[x] = max(pref[2 * x], total[2 * x] + pref[2 * x + 1])

    def build(x, l, r):
        if r - l == 1:
            total[x] = pref[x] = a[l]
            return
        m = (l + r) // 2
        build(2 * x, l, m)
        build(2 * x + 1, m, r)
        pull(x)

    def update(x, l, r, p, v):
        if r - l == 1:
            total[x] = pref[x] = v
            return
        m = (l + r) // 2
        if p < m:
            update(2 * x, l, m, p, v)
        else:
            update(2 * x + 1, m, r, p, v)
        pull(x)

    def first(x, l, r, ql, budget, running):
        if r <= ql:
            return (-1, running)
        if ql <= l and running + pref[x] <= budget:
            return (-1, running + total[x])
        if r - l == 1:
            return (l, running)
        m = (l + r) // 2
        answer, running = first(2 * x, l, m, ql, budget, running)
        if answer >= 0:
            return answer, running
        return first(2 * x + 1, m, r, ql, budget, running)
    build(1, 0, n)
    j = 2 + n
    out = []
    for _ in range(q):
        c = d[j]
        i = int(d[j + 1]) - 1
        x = int(d[j + 2])
        j += 3
        if c == b'U':
            update(1, 0, n, i, x)
        else:
            answer, _ = first(1, 0, n, i, x, 0)
            out.append(str(answer + 1 if answer >= 0 else -1))
    print('\n'.join(out))
if __name__ == '__main__':
    main()
```

17 L. Range Add Threshold Search

17.1 Lazy maximum tree

Store the maximum value of each segment. A range addition increases both the stored maximum and the lazy tag of every fully covered node by the same amount. Before entering a partially covered node, push its tag to its two children.

For `Q l r x`, a node can be discarded if it is outside `[l,r]` or its maximum is less than `x`. Otherwise, if it is not a leaf, push its lazy tag and search the left child before the right child. The first reachable leaf is the smallest valid position.

17.2 Correctness and cost

Every node maximum equals the greatest current array value in its segment: range addition preserves the invariant by shifting all covered values and the stored maximum equally, and pulling takes the maximum of the children.

A discarded in-range node contains no qualifying value because its maximum is below the threshold. Every node before the query interval is also discarded. Searching left before right therefore returns the smallest qualifying index, or `-1` when the root search finds none.

Both range addition and guided descent take `O(log n)` time. Building takes `O(n)` time and the tree uses `O(n)` memory.

17.3 Full solutions

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

struct SegmentTree {
  int n;
  vector<ll> maximum, lazy;

  explicit SegmentTree(const vector<ll> &values)
      : n(values.size()), maximum(4 * n), lazy(4 * n) {
    build(1, 0, n, values);
  }

  void build(int node, int left, int right, const vector<ll> &values) {
    if (right - left == 1) {
      maximum[node] = values[left];
```

```
    return;
  }
  int middle = (left + right) / 2;
  build(2 * node, left, middle, values);
  build(2 * node + 1, middle, right, values);
  pull(node);
}

void pull(int node) {
  maximum[node] = max(maximum[2 * node], maximum[2 * node + 1]);
}

void apply(int node, ll delta) {
  maximum[node] += delta;
  lazy[node] += delta;
}

void push(int node) {
  if (lazy[node] == 0)
    return;
  apply(2 * node, lazy[node]);
  apply(2 * node + 1, lazy[node]);
  lazy[node] = 0;
}

void add(int node, int left, int right, int query_left, int query_right,
         ll delta) {
  if (right <= query_left || query_right <= left)
    return;
  if (query_left <= left && right <= query_right) {
    apply(node, delta);
    return;
  }
  push(node);
  int middle = (left + right) / 2;
  add(2 * node, left, middle, query_left, query_right, delta);
  add(2 * node + 1, middle, right, query_left, query_right, delta);
  pull(node);
}

int first_at_least(int node, int left, int right, int query_left,
                   int query_right, ll threshold) {
  if (right <= query_left || query_right <= left ||
      maximum[node] < threshold)
    return -1;
  if (right - left == 1)
    return left;
  push(node);
  int middle = (left + right) / 2;
  int answer = first_at_least(2 * node, left, middle, query_left,
                              query_right, threshold);
```

```
    if (answer != -1)
      return answer;
    return first_at_least(2 * node + 1, middle, right, query_left,
                          query_right, threshold);
  }
};

int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);

  int n, q;
  if (!(cin >> n >> q))
    return 0;

  vector<ll> values(n);
  for (ll &value : values)
    cin >> value;

  SegmentTree tree(values);
  while (q--) {
    char operation;
    int left, right;
    ll x;
    cin >> operation >> left >> right >> x;
    --left;

    if (operation == 'A') {
      tree.add(1, 0, n, left, right, x);
    } else {
      int answer = tree.first_at_least(1, 0, n, left, right, x);
      cout << (answer == -1 ? -1 : answer + 1) << '\n';
    }
  }
}
```

```
import sys

def main():
    d = sys.stdin.buffer.read().split()
    if not d:
        return
    n, q = map(int, d[:2])
    a = list(map(int, d[2:2 + n]))
    maximum = [0] * (4 * n)
    lazy = [0] * (4 * n)

    def build(node, left, right):
        if right - left == 1:
            maximum[node] = a[left]
            return
```

```
        middle = (left + right) // 2
        build(2 * node, left, middle)
        build(2 * node + 1, middle, right)
        maximum[node] = max(maximum[2 * node], maximum[2 * node + 1])

    def apply(node, delta):
        maximum[node] += delta
        lazy[node] += delta

    def push(node):
        delta = lazy[node]
        if delta:
            apply(2 * node, delta)
            apply(2 * node + 1, delta)
            lazy[node] = 0

    def add(node, left, right, query_left, query_right, delta):
        if right <= query_left or query_right <= left:
            return
        if query_left <= left and right <= query_right:
            apply(node, delta)
            return
        push(node)
        middle = (left + right) // 2
        add(2 * node, left, middle, query_left, query_right, delta)
        add(2 * node + 1, middle, right, query_left, query_right, delta)
        maximum[node] = max(maximum[2 * node], maximum[2 * node + 1])

    def first(node, left, right, query_left, query_right, threshold):
        if (right <= query_left or query_right <= left
                or maximum[node] < threshold):
            return -1
        if right - left == 1:
            return left
        push(node)
        middle = (left + right) // 2
        answer = first(
            2 * node, left, middle, query_left, query_right, threshold
        )
        if answer != -1:
            return answer
        return first(
            2 * node + 1, middle, right, query_left, query_right, threshold
        )

    build(1, 0, n)
    out = []
    p = 2 + n
    for _ in range(q):
        operation = d[p]
        left, right, x = map(int, d[p + 1:p + 4])
```

```
        p += 4
        left -= 1
        if operation == b'A':
            add(1, 0, n, left, right, x)
        else:
            answer = first(1, 0, n, left, right, x)
            out.append(str(answer + 1 if answer != -1 else -1))
    print('\n'.join(out))
if __name__ == '__main__':
    main()
```